UNIVERSITY of SAN CARLOS

SCIENTIA • VIRTUS • DEVOTIO

Subprograms

CS3204N Programming Languages

What is a Subprogram?

A **subprogram** is a reusable block of code that performs a specific task

Two main types:

- **Function** → returns a value
- **Procedure** → does not return a value

Simple Example (C)

```
int add(int a, int b) {
        return a + b;
}

int main() {
        int result = add(3, 5);
        printf("%d", result);
}
```

Execution Flow

- Main program calls subprogram
- Control transfers to subprogram
- Subprogram executes
- Control returns to caller

Call stack: main → add → return → main

Why Use Subprograms?

- Avoid repetition
- Improve readability
- Enable modular design

Subprogram Structure

Header:

- Name
- Parameters
- Return type

Body:

- Statements

Formal vs Actual Parameters

```
int multiply(int x, int y) {  // formal parameters
    return x * y;
}

multiply(4, 6);  // actual parameters
```

Return Values (Multiple Values Example - Python)

```
def compute(a, b):
        return a + b, a * b

sum_val, prod_val = compute(2, 3)
```

Side Effects Example

```
int x = 10;

void modify() {
        x = 20; // side effect
}
```

Local Variables

```
void func() {
        int x = 5; // local
}
```

Global vs Local

```
int x = 10;

void func() {
        int x = 5;
        printf("%d", x);
}
```

Static Scope

Variable is resolved based on **where it is written in code.**

```
int x = 10;

void func() {
        printf("%d", x);
}
```

Dynamic Scope

Variable is resolved based on **who called the function**

```
function A() {
x = 10
B()
}

function B() {
print(x)
}
```

Pass-by-Value

A copy of the value is passed

```
void change(int x) {
        x = 50;
}

int main() {
        int a = 10;
        change(a);
        printf("%d", a); // 10
}
```

Memory Illustration

- a = 10 → copied to x
- x changes → a unaffected

Pass-by-Reference

Address of variable is passed

```
void change(int &x) {
        x = 50;
}
```

Output becomes: 50

Swap Example

```
void swap(int &a, int &b) {
        int temp = a;
        a = b;
        b = temp;
}
```

Pass-by-Result

```
procedure add(out x)
        x = 10
```

Pass-by-Value-Result

Copy in → modify → copy out

Pass-by-Name

Expression is evaluated every time it is used.

```
procedure example(x)
        print(x)
```

Call:

```
        example(a + b)
```

Comparison Table

Method	Input	Output	Side Effects
Value	Yes	No	No
Reference	Yes	Yes	Yes
Result	No	Yes	Yes
Value-Result	Yes	Yes	Yes

Example

```
def apply(func, x):
        return func(x)

def square(n):
        return n * n

print(apply(square, 5))
```

Function Pointer

Variable that stores a function

```
int add(int a, int b) {
        return a + b;
}

int (*funcPtr)(int, int) = add;
funcPtr(2, 3);
```

Pure Function

No side effects.

```
def add(a, b):
        return a + b
```

Impure Function

Output may change due to external variables

```
x = 10

def change():
        global x
        x = 20
```

Impure Function

```
x = 10

def change():
        global x
        x = 20
```

Overloading

```
int add(int a, int b);
double add(double a, double b);
```

Generics

Function works for many data types.

C++ Template Example

```
template <typename T>
T add(T a, T b) {
    return a + b;
}
```

Operator Overloading

Operators behave differently for objects

```
class Point {
public:
        int x, y;
        Point operator+(Point p) {
                return {x + p.x, y + p.y};
        }
};
```

Closures

Inner function “remembers” x

```
function outer() {
        let x = 10;
        return function inner() {
                return x;
        };
}
```

Coroutines

Function pauses and resumes

Python Generator Example

```
def counter():
    yield 1
    yield 2
    yield 3
```

Coroutine Behavior

Pause → Resume → Continue